PLANORA

Gestión de tareas y hábitos

Documento de arquitectura · Propuesta de prototipo móvil

Versión académica · 22 de octubre de 2026

Objetivo

Diseñar una experiencia móvil simple para organizar tareas, calendario y hábitos con un sistema visual consistente, modo claro/oscuro y navegación de baja fricción.

Estado de la entrega: este PDF y el paquete Flutter son artefactos iniciales. El HTML incluido es un prototipo navegable de referencia; no es un archivo Figma ni un enlace público de Figma.

1. Alcance funcional

Pantallas requeridas: Login, Dashboard, Crear tarea, Calendario y Perfil. El flujo principal es Login → Dashboard → Crear tarea → Dashboard. Desde la barra inferior se accede a Calendario y Perfil. El prototipo HTML permite cambiar entre pantallas y alternar tema claro/oscuro.

Componentes reutilizables

Componente	Uso
Botón primario	Acción principal: iniciar sesión, guardar tarea.
Campo de formulario	Correo, contraseña, título, descripción y fecha.
Tarjeta	Resumen diario, tarea, racha de hábito.
Chip	Categoría, prioridad o estado.
Barra inferior	Acceso persistente a Inicio, Calendario, Crear y Perfil.
Indicador de progreso	Porcentaje de tareas completadas.

2. UML — Casos de uso

Actores/roles: Visitante, Usuario registrado, Administrador y Servicio de notificaciones. El servicio es un actor externo que entrega recordatorios.

```
@startuml
left to right direction
actor Visitante
actor "Usuario registrado" as Usuario
actor Administrador
actor "Servicio de notificaciones" as Push
rectangle Planora {
  usecase "Registrarse / iniciar sesión" as UC1
  usecase "Gestionar tareas" as UC2
  usecase "Gestionar hábitos" as UC3
  usecase "Consultar calendario" as UC4
  usecase "Editar perfil y preferencias" as UC5
  usecase "Administrar categorías" as UC6
  usecase "Enviar recordatorios" as UC7
}
Visitante --> UC1
Usuario --> UC2
Usuario --> UC3
Usuario --> UC4
Usuario --> UC5
Administrador --> UC6
Push --> UC7
UC2 .> UC6 : <<usa>>
UC7 .> UC2 : <<lee vencimientos>>
@enduml
```

3. UML — Diagrama de clases

```
@startuml
class Usuario {
  +id: String
  +nombre: String
  +correo: String
  +preferenciaTema: String
}
class Tarea {
  +id: String
  +titulo: String
  +descripcion: String
  +fechaVencimiento: DateTime
  +estado: EstadoTarea
  +prioridad: Prioridad
}
class Categoria {
  +id: String
  +nombre: String
```

```
  +colorHex: String
}
class Notificacion {
  +id: String
  +mensaje: String
  +programadaPara: DateTime
  +leida: Boolean
}
Usuario "1" -- "0..*" Tarea : crea
Usuario "1" -- "0..*" Notificacion : recibe
Categoria "1" -- "0..*" Tarea : clasifica
Tarea "1" -- "0..*" Notificacion : genera
@enduml
```

4. Arquitectura seleccionada: Clean Architecture + MVVM

Se propone Clean Architecture para separar dominio, acceso a datos y presentación, usando MVVM en la capa de presentación. Es adecuada porque el proyecto puede crecer desde un prototipo a persistencia local, sincronización en nube y notificaciones sin acoplar las pantallas a una base de datos concreta.

Capas

Presentation: vistas y ViewModels; gestiona estado de pantalla y eventos de usuario. **Domain:** entidades y contratos de repositorio; contiene reglas de negocio futuras. **Data:** implementaciones de repositorios y fuentes de datos locales/remotas. **Core:** tema, configuración y registro de dependencias compartidas.

Dirección de dependencias

```
Presentation  --->  Domain  <---  Data
       |                          |
       +------ Core / DI --------+
Domain no depende de Flutter, API ni almacenamiento.
```

Ventajas: facilita pruebas unitarias, reemplazo de almacenamiento, mantenimiento y colaboración. Coste: para una app pequeña añade carpetas y abstracciones; se justifica aquí por el objetivo académico de demostrar arquitectura escalable.

5. Patrones de diseño

Repository: el contrato TaskRepository abstrae la obtención y guardado de tareas. La UI no conoce si los datos vienen de SQLite, Supabase o una API. La implementación de datos se puede sustituir sin cambiar el ViewModel.

Singleton: un punto de composición/ServiceLocator único puede centralizar instancias compartidas de repositorios y servicios. Debe usarse con moderación para evitar dependencias globales difíciles de probar; en producción se recomienda inyección de dependencias explícita.

Observer (complementario): el ViewModel notifica cambios de estado a la vista; al crear o completar una tarea, la UI reacciona sin consultar manualmente cada componente.

6. Guía de estilo propuesta

Token	Valor	Uso
Primary	#635BFF	Botones principales, enlaces, selección
Secondary	#20BFA9	Progreso y estados positivos
Light background	#F7F7FC	Fondo modo claro
Surface	#FFFFFF	Tarjetas modo claro
Dark background	#151622	Fondo modo oscuro
Dark surface	#222333	Tarjetas modo oscuro
Text	#202033 / #F5F5FF	Texto principal claro / oscuro
Muted	#77788E / #AAAA BD	Texto secundario claro / oscuro

Tipografía: Inter. Títulos 24–28 px, subtítulos 18–20 px, cuerpo 14–16 px, etiquetas 12–13 px. Usar pesos 400 para cuerpo, 600 para subtítulos y 700–800 para encabezados.

Forma y espaciado: tarjetas con radio de 18–20 px; campos y botones con radio de 12–14 px; escala de espaciado basada en múltiplos de 4 px; margen horizontal móvil de 20 px.